

Problem 5.3**Solution:**

- (a) Use a uniform grid to efficiently check for point pairs within distance r . Divide the plane into a uniform grid where each cell is a square of size $(r/2) \times (r/2)$. If any cell contains two or more points, their distance is necessarily less than r . If a point has no neighbor in its own cell, it only needs to check points in adjacent cells (at most 24 cells since the side length of the square is $r/2$). Use a hash table to store the occupied grid cells, ensuring $O(1)$ insertion and lookup in expectation.

Algorithm 1

```

1: Initialize a hash table  $T$ 
2: for each point  $p = (x, y) \in P$  do
3:    $(v, w) = \left( \left\lfloor \frac{x}{r/2} \right\rfloor, \left\lfloor \frac{y}{r/2} \right\rfloor \right)$ 
4:   Transform and combine  $v$  and  $w$  into a 0-1 vector:  $\{0, 1\}^{2\log_2 U}$ , as the index of  $p$ .
5:   Store  $p$  in the hash table  $T$ :
6:   if another point already exists in the cell then
7:     return True
8:   else
9:     Check all 24 neighboring cells (a  $5 \times 5$  grid centered at  $p$ ). If any existing point in these
       cells is within distance  $r$ , return True.
10: return False

```

Time complexity analysis:

For each point, computing the grid index takes $O(1)$; Hash table operations (insertion and lookup) take expected $O(1)$; Checking adjacent cells involves at most a constant number of comparisons, also $O(1)$. Thus, for n points, the expected total runtime is $O(n)$.

- (b) The algorithm correctly finds the closest pair distance $\delta(P)$ by employing a divide-and-conquer strategy with a randomized approach.

Base Case: If $|P| \leq 100$, the algorithm directly computes the closest pair using brute force, which is guaranteed to be correct.

Partitioning: The set P is divided into 20 subsets $P_1, P_2, \dots, P_{20}$, each containing at most $\lceil n/20 \rceil$ points.

The algorithm considers all pairs of subsets from the set

$$S = \{(i, j) \mid 1 \leq i < j \leq 20\}.$$

Since there are 20 subsets, the number of pairs is:

$$|S| = \binom{20}{2} = 190.$$

The algorithm iterates over the pairs (i, j) in random order. If $\delta(P_i \cup P_j) < r$, it updates r with the result of a recursive call to $\text{ClosestPair}(P_i \cup P_j)$. The global closest pair distance $\delta(P)$ must exist in at least one of these subset pairs. Thus, as long as the correct pair (i^*, j^*) is considered, the algorithm will correctly compute $\delta(P)$.

The decision algorithm from part (a) is used to check whether $\delta(P_i \cup P_j) < r$ before making recursive calls.

Since the algorithm always considers a valid pair that contains the closest points, it will return the correct answer.

Time complexity analysis:

Let $T(n)$ denote the expected runtime of the algorithm. The partitioning step (line 2) takes $O(n)$ time. There are 190 pairs, and each pair is checked in constant time (except for potential recursive calls).

For each pair (i, j) , the decision problem (from part (a)) runs in $O(n)$ expected time. Since the subset $P_i \cup P_j$ has at most $O(n/10)$ points, the decision step per pair runs in $O(n)$. However, not all 190 pairs will result in recursive calls. Since the pairs are traversed in a random order, the optimal pair (i^*, j^*) appears with equal probability at some position within the first k iterations. Once the correct minimum distance $\delta(P)$ is found, subsequent pairs will not trigger recursive calls. Thus the expected number of times line 7 (recursive call) executed is a constant.

Therefore, in expectation, the algorithm makes a recursive call on a subset of at most $O(n/10)$ points, a constant number of times.

The expected runtime satisfies the recurrence:

$$T(n) = O(n) + O(1) \cdot T(n/10).$$

Expanding the recurrence:

$$T(n) = O(n) + T(n/10).$$

Applying the recurrence iteratively:

$$T(n) = O(n) + O(n/10) + O(n/10^2) + O(n/10^3) + \dots$$

This forms a geometric series:

$$O \left(n \sum_{k=0}^{\log_{10} n} \frac{1}{10^k} \right).$$

Since the sum of the geometric series converges to a constant, we get:

$$T(n) = O(n).$$